

RallyPoint

Milestone 2 演示视频脚本 · Demo Video Script (Backend Component)

总时长约 9:00

英文口播约 1,330 词

覆盖：API List + Test Docs /20 · Database Design /20 · API Testing + Live Demo /60

本里程碑 = ACIS 498 Milestone 2 后端 (Flask 3 + SQLAlchemy + JWT + Pydantic v2)

全程真实代码与真实运行结果，无夸大

数字均已实测核对：60 endpoints · 17 tables · 124 tests · 39-call live demo

开场 OPENING | 0:00 – 0:45

【画面】浏览器打开 `http://localhost:5050/api/health` 显示 healthy JSON；旁边一个标签页停在 GitHub 仓库首页。

Hi, I'm Phoebe. This is Milestone 2 of my capstone, RallyPoint — an AI-powered partner-matching app for everyday tennis and pickleball players. Milestone 1 was the React front end; today is the backend it talks to: a Flask REST API with sixty endpoints over a seventeen-table SQLAlchemy database, JWT auth, Pydantic validation, and an explainable AI matching service.

【画面】切到 VS Code，短暂停在 `backend/` 目录树。

Here's the plan: the architecture, the database design, the API live in Swagger, then the heart of this assignment — proof that the database updates after every single operation — then the automated tests, and two real issues I hit and fixed.

(约 115 词)

架构 ARCHITECTURE | 0:45 – 1:30

【画面】打开 `backend/app.py`，指 `create_app()` 和 `register_blueprints()`；再开 `routes/__init__.py` 让 9 个 blueprint 一屏可见。

The app is an application factory. Nine blueprints — auth, players, sessions, ai, courts, appointments, invites, admin, and support — each owning one slice of the API, fifty-nine routes between them plus a health check.

【画面】打开 `extensions.py` (db / migrate / limiter 三个单例)。

Shared extensions — the database, migrations, the rate limiter — live in their own module with no app attached, and the factory binds them at startup. That one-file pattern matters; it comes back in the issues section. Every write body is validated by a Pydantic schema before any handler runs, and auth is a JWT in an httpOnly cookie with CSRF for browsers, or a Bearer header for API clients.

(约 115 词)

数据库设计 DATABASE DESIGN | 1:30 – 3:00 | /20 □

【画面】打开 milestone2/MILESTONE_2_DATABASE_DESIGN.md 的 ER 图整体停 3 秒，然后切到 models/ 目录。

The schema is seventeen tables. Rather than read them all, let me call out the design choices I'm proud of.

【画面】models/user.py — 指 sport_profiles。

Profiles are per sport — so one person can be a 4.0 tennis player and a 3.0 pickleball player, each with its own rating and preferences.

【画面】models/game_invite.py — 指 game_invites 和 time_proposals 两个类。

Game invites are two-phase: first you agree to play, then you negotiate the time through proposal rows — and only when a time locks does the system materialize a real row in the sessions table. You'll see that happen live in the database in a minute.

【画面】models/ai_match_log.py — 指 unique constraint。

AI match verdicts are cached with a unique constraint on viewer, candidate, and sport — calling the endpoint twice upserts instead of duplicating.

【画面】models/availability.py — 上下两个类: availability_slots 与 availability_overrides。

Availability is a weekly grid, plus the newest table — dated overrides that layer date-specific exceptions on top: "normally free Tuesday evenings, but not on the 4th."

【画面】models/user.py 拉到 UserPhoto。

And profile photos are stored right in the database — one row per user, a client-resized thirty-kilobyte JPEG blob. At this scale that's deliberate: no S3 bucket, no orphaned files, one backup story. Everything runs on SQLite in dev and PostgreSQL in production on Render — same ORM models, one DATABASE_URL switch, zero code change.

(约 235 词)

Swagger 实时演示 LIVE API | 3:00 – 4:20 | /20 □□

【画面】切到 <http://localhost:5050/api/docs/>，慢慢滚动按 tag 分组的端点列表。

Every endpoint is self-documented in Swagger with expected input and output — this is the interactive version of my API reference doc, which lists all sixty routes with sample JSON for each.

【画面】展开 POST /auth/login → Try it out → 输入 {"email": "alex@rally.app", "password": "rally1234"} → Execute → 指返回的 token。

Let's use it for real. I log in as a seeded user — back comes the user object and a JWT.

【画面】点右上 Authorize，粘贴 Bearer <token>；然后执行 GET /auth/me。

I authorize the session with that token, and /auth/me confirms who I am.

【画面】执行 GET /players?sport=Pickleball，滚动到一条结果，指 matchScore / matchTier / matchReasons。

And here's the AI matching endpoint the front end is built on: candidates ranked by a transparent 0-to-100 score — skill closeness, real Haversine distance, schedule overlap, shared courts, plus an optional embedding-based playing-style signal — every contributor emitting a human-readable reason chip. Nothing is a black box.

(约 185 词)

校验与安全 VALIDATION & SECURITY | 4:20 – 5:00

【画面】Swagger 里执行 POST /auth/signup, body 填 {"email": "not-an-email", "password": "short", "name": ""}, 展示结构化 422 响应的 per-field errors。

What about bad input? A junk signup never crashes the API — Pydantic returns a structured 422 with a message per field, so the client knows exactly what to fix. On top of that: passwords are script-hashed, login is rate-limited against brute force, the JWT lives in an httpOnly cookie with CSRF on writes, and a spoofed X-User-Id header is simply rejected — there's an automated test for exactly that attack.

(约 90 词)

数据库实时证据 LIVE DB EVIDENCE | 5:00 – 7:00 | /60 □□□ 核心段

【画面】切到终端: `cd backend && .venv/bin/python api_demo.py | less`, 先停在开头的说明块。

Now the core requirement: evidence that the database updates after each REST operation. This script drives the real Flask app end to end — thirty-nine real API calls against a fresh database — and after every single one it prints what changed: row-count deltas for inserts and deletes, and the exact before-and-after field values for in-place updates.

【画面】翻到 STEP 1 (signup), 指 DB DELTA: users 2->3 (+1 row); sport_profiles 2->3 (+1 row)。

A signup — and the users table goes from two rows to three, with a sport profile created alongside.

【画面】翻到 STEP 7 (dated availability override), 指 availability_overrides 0->1 (+1 row) 和 ROW AFTER 的日期行。

A date-specific availability override — one of the newest features — lands as its own row layered over the weekly grid: this player is busy on that one morning only.

【画面】翻到 STEP 8 (photo upload), 指 user_photos 0->1 (+1 row) 和 ROW AFTER 里的 mime / bytes。

Uploading a profile photo inserts the blob row — you can read the mime type and the byte size straight out of the database.

【画面】翻到 STEP 17 (Maya accepts), 指 ROW BEFORE status=pending / ROW AFTER status=confirmed。

Accepting a game request is an in-place update — same row, status pending before, confirmed after.

【画面】翻到 STEP 26 – 27 (join full game → waitlisted; leave → promoted)。

Joining a full open game adds a participant as waitlisted; when someone leaves, the next person is promoted automatically — both visible in the participants table.

【画面】翻到 STEP 32 (accept time → materialize) , 指 sessions +1 和 invite 行的 phase → confirmed, session_id。

Here's the two-phase invite paying off: accepting a proposed time inserts a sessions row and stamps the invite with its id — the materialization I showed you in the schema.

【画面】翻到 STEP 37 (review report + suspend) : report status → reviewed, user is_active 1 → 0。

Moderation: an admin reviews a report and suspends the offender — the report flips to reviewed and the user's is_active goes from one to zero, in one transaction.

【画面】翻到 STEP 38 (admin 删头像) , 指 user_photos 1->0 (-1 row); 再滚到最底部, 停在 Total API calls exercised: 39 和 17 张表的 FINAL DB STATE。

The admin strips that user's photo too — minus one row, back to the initials avatar. Thirty-nine operations, thirty-nine pieces of database evidence, ending with the final state of all seventeen tables. The full transcript is committed in the repo as api_demo_output.txt.

(约 310 词)

自动化测试 AUTOMATED TESTS | 7:00 – 7:45

【画面】终端运行 `pytest -v` (可加速快进) , 最后停在 124 passed 汇总行 3 秒。

Behind the live demo is the automated suite: one hundred twenty-four tests across fourteen files, all passing. They cover the whole surface — the auth lifecycle including token revocation, the invite state machine, matching math, admin guards — and the security invariants: header spoofing rejected, suspended users hidden, admin rights not grantable over the API. Every test runs against a fresh in-memory database, so they're fully isolated, and the suite runs in CI on every push.

(约 90 词)

问题与解决 ISSUES & RESOLUTIONS | 7:45 – 8:45

【画面】打开 `extensions.py`, 旁边开 `milestone2/issue_diagnosis_1_flask_limiter_circular_import.md` 的 import-graph 小图。

Two real issues, both documented in the repo. First, when I added rate limiting to login, the app died with a circular import: the Limiter lived in `app.py`, and any blueprint that wanted it had to import from `app.py` while `app.py` was still importing that blueprint. I sketched the import graph, then moved every shared singleton into a dedicated extensions module that depends on nothing — the factory binds them later. The cycle is gone for good, not patched around.

【画面】打开 `milestone2/issue_diagnosis_2_pydantic_python314_py03.md`, 指 `pip download --only-binary` 那行验证命令, 再切 `requirements.txt` 的 `pydantic[email]>=2.11.0`。

Second, on a fresh venv, pydantic-core refused to install on Python 3.14 — its Rust layer, PyO3, capped out at 3.12, and no prebuilt wheel existed. I confirmed it was a wheel-availability problem, not my toolchain, with `pip download only-binary`, then pinned Pydantic to 2.11, which ships Python-3.14 wheels. Lesson: on a fresh Python, pin with the wheel matrix in mind — and verify the fix with the test suite, which is exactly what the 422 validation tests are for.

(约 175 词)

结尾 CLOSING | 8:45 – 9:10

【画面】回到 Swagger 端点总览，最后切一眼 124 passed。

So that's the Milestone 2 backend: sixty endpoints across nine blueprints, a seventeen-table relational schema, JWT auth, Pydantic validation, explainable AI matching — fully integrated with the database, proven by a per-operation live demo and a hundred twenty-four passing tests, and already live in production on Render. Next up: connecting front and back in the final integrated demo. Thanks for watching.

(约 65 词)

英文口播合计约 1,370 词 (115+115+235+185+90+300+90+175+65)；按 ~150 词/分钟约 9:08，在 6 – 10 分钟要求内。若现场偏长，优先压缩 Swagger 段的 /auth/me 一步和 Tests 段的枚举；勿压 DB EVIDENCE 段（占 60 分）。

录制小贴士

- 录前准备（一次做完）：`cd backend && source .venv/bin/activate && python seed.py`（重置演示数据：6 users / 8 courts / 6 sessions，□□ 会清空本地开发库）→ `python app.py`（跑在 5050 端口）。浏览器开两个标签：/api/health 和 /api/docs/；VS Code 按出场顺序排好标签：`app.py` → `routes/__init__.py` → `extensions.py` → `models/user.py` → `game_invite.py` → `ai_match_log.py` → `availability.py` → `requirements.txt`，外加两篇 issue 诊断 md 和 MILESTONE_2_DATABASE_DESIGN.md（停在 ER 图）。第二个终端在 backend/ 目录待命跑 `api_demo.py` 和 `pytest -v`。
- 数字务必念新版：60 endpoints（59 blueprint 路由 + health）、17 tables、124 tests、api_demo 39 calls——旧文档里的 55/15/120/36 已全部更新，别念错。
- DB EVIDENCE 段（60 分核心）：`| less` 翻页比拖滚动条稳；每停一处，鼠标先指 API 调用行、再指 DB DELTA/ROW BEFORE/AFTER 行，念完再翻页。作业原文要求 "evidence of the database update after EACH operation"——这一段每个例子都要把「操作 → 数据库变化」指给屏幕看。
- Swagger 段：login 用种子账号 `alex@rally.app / rally1234`；Authorize 时记得贴 Bearer 前缀。若 matchReasons 里没有 "similar playing style" chip（本地没设 OPENAI_API_KEY 就不会有），只说架构支持 embedding 信号、不要声称画面上有这个 chip；/api/ai/match-reason 的 source 会是 "heuristic"，这是优雅降级、可以坦然讲。
- 诚实边界：日期覆盖（availability overrides）没有独立端点，是搭在 PATCH /api/auth/me 上的——别说 "override endpoint"；在线 LLM 只重写措辞且默认关闭；头像存 DB blob 是有意为之（规模小、免 S3），照脚本里的理由讲。
- pytest 段：`pytest -v` 约 35 秒，可以先录全程再在剪辑里 2× 快进，只要最后的 124 passed 清晰停留即可；或改跑 `pytest -q`（十几秒出结果）。
- 时长控制：目标 9 分上下，硬上限 10 分。超时先砍 Swagger 的 /auth/me 演示和 Closing 的展望句。
- 全程只展示真实代码与真实运行输出；两篇 issue 文档是现成的，讲述时与文档一致即可。